

1. Objectives:

1. To learn the concept of Array & Vector (STL).
2. To understand how to find the maximum number from a 2D vector.
3. To understand how to do the sum of sub array.
4. To learn the concept of Sliding Window of an array.

2. Introduction:

- **Array:** An array is a collection of similar data elements stored in consecutive memory locations under a single variable name. Each element in an array can be accessed directly using its index, and the size of the array is fixed at the time of declaration. Arrays are used to build other data structures like Stack Queue, Deque, Graph, Hash Table, etc.

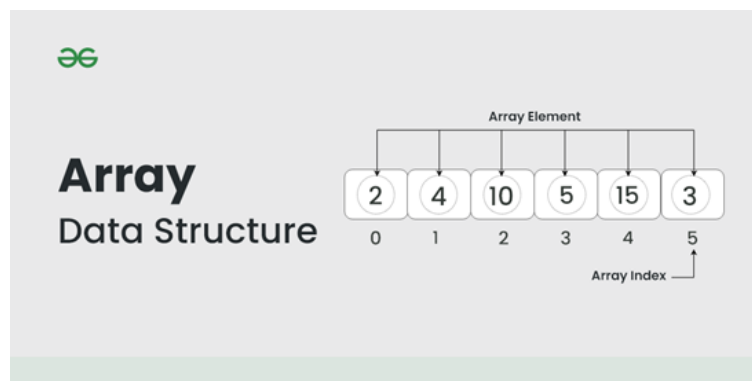


Fig:1.1: Array

- **Vector:** Vector is a dynamic array provided by the Standard Template Library (STL). Unlike normal arrays, vectors can change their size dynamically during program execution. Vectors support various built-in functions such as `push_back()`, `insert()`, `erase()`, and allow easy access to elements.
- **2D Vector:** 2D vector is a vector of vectors, it can be visualized as a matrix where each inner vector represents a row, and the number of rows represents the maximum columns. The advantage over 2D arrays is, we can have dynamically sized rows and columns, i.e., every row can have different numbers of columns flexible in size and no extra parameters required while passing to function.

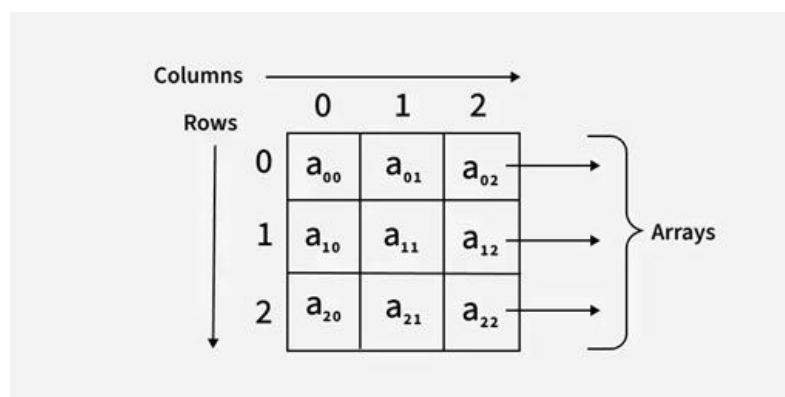


Fig:1.2: 2D vector

- **Sliding Window:** Sliding Window Technique is a method used to solve problems that involve subarray, substring, or window. It is an efficient method used to process a fixed-size window over an array by moving it step by step and performing calculations on each window.

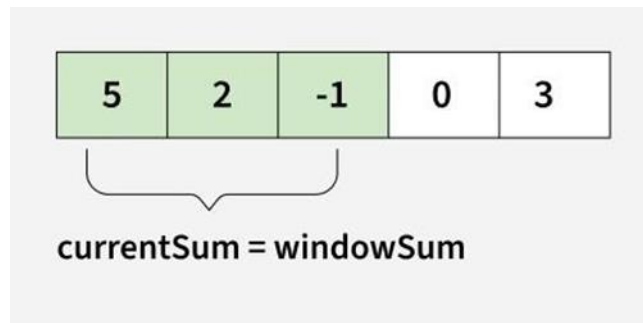


Fig:1.3: Sliding Window

3. Procedure

Experiment 1: System of vector / Introduction of vector.

Input:

- No input parameters from user

Output:

- Prints the values of the vector after each major stage

Algorithm:

1. Create an empty vector v of integers
2. **push_back** 5 into v
3. **push_back** 10 into v
4. **push_back** 15 into v
5. **push_back** 20 into v
6. **push_back** 25 into v
7. **for** $i \leftarrow 0$ to $\text{size}(v) - 1$ **do**
 print $v[i]$
8. **insert** value at index position in v
9. **for** $i \leftarrow 0$ to $\text{size}(v) - 1$ **do**
 print $v[i]$
10. **erase** element at index 2 from v
11. **for** each element a in v **do**
 print a
12. **return** v

Experiment 2: Finding the maximum number from a 2D vector

Input:

- v – A 2D vector (3×3 matrix)

- r – Row index
- c – Column index
- k – Integer value inserted into the matrix

Output:

- max – Maximum element of the 2D vector

Algorithm:

1. Create an empty 2D vector v
2. **for** $r \leftarrow 0$ to 2 **do**
3. Create an empty vector $v1$
 - for** $c \leftarrow 0$ to 2 **do**
 - Read integer k
 - push_back** k into $v1$
 - push_back** $v1$ into v
4. $max \leftarrow INT_MIN$
5. **for** $r \leftarrow 0$ to 2 **do**
 - for** $c \leftarrow 0$ to 2 **do**
 - if** $v[r][c] > max$ **then**
 - $max \leftarrow v[r][c]$
6. **print** max

Experiment 3: Finding the Sum of a Subarray.

Input:

- v – A vector containing integer elements
- $startposition$ – Starting index of the subarray
- $endposition$ – Ending index of the subarray

Output:

- sum – Sum of elements from $startposition$ to $endposition$

Algorithm:

1. $sum \leftarrow 0$
2. **for** $i \leftarrow startposition$ to $endposition$ **do**
 - $sum \leftarrow sum + v[i]$
3. **print** sum

Experiment 4: Finding max of sliding window from array.

Input:

- $a[]$ – A given array of 6 integers

Output:

- max – Maximum sum among all subarrays of size k

Algorithm:

```

1. max ← INT_MIN
2. for i ← 0 to 3 do
    sum ← 0
    for j ← 0 to 2 do
        sum ← sum + a[i + j]
    if sum > max then
        max ← sum
3. print max

```

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: System of vector / Introduction of vector.

```

#include <bits/stdc++.h>
using namespace std;

int main(){
    vector<int> v;
    v.push_back(5);
    v.push_back(10);
    v.push_back(15);
    v.push_back(20);
    v.push_back(25);
    for (int i = 0; i < v.size(); i++){
        cout << v[i] << " ";
    }
    int position = 2;
    int value = 100;
    v.insert(v.begin() + position, value);
    for (int i = 0; i < v.size(); i++){
        cout << v[i] << " ";
    }
    v.erase(v.begin() + 2);
    cout << endl;
    for (int a : v){
        cout << a << " ";
    }
    return 0;
}

```

Table 1.1: Experiment 1 output

```

5 10 15 20 25 5 10 100 15 20 25
5 10 15 20 25

```

Experiment 2: Finding the maximum number from a 2D vector

```

#include <bits/stdc++.h>
using namespace std;

int main(){
    cout << "Enter 9 numbers: ";
    int k;
    vector<vector<int>> v;
    for (int r = 0; r < 3; r++){
        vector<int> v1;

        for (int c = 0; c < 3; c++){
            cin >> k;
            v1.push_back(k);
        }
        v.push_back(v1);
    }
    int max = INT_MIN;
    for (int r = 0; r < 3; r++){
        for (int c = 0; c < 3; c++){
            if (v[r][c] > max)
                max = v[r][c];
        }
    }
    cout << max << endl;
    return 0;
}

```

Table 1.2: Experiment 2 output

Input	Output
Enter 9 numbers: 1 2 3 4 5 6 7 8 9	9

Experiment 3: Finding the Sum of a Subarray.

```

#include <bits/stdc++.h>
using namespace std;

int main()
{
    int sum = 0;
    vector<int> v = {5, 10, 15, 20, 25};

    int startposition, endposition;
    cout << "Enter Start Position and End Position: ";
    cin >> startposition >> endposition;

    for (int i = startposition; i <= endposition; i++)
    {
        sum = sum + v[i];
    }
    cout << sum;

    return 0;
}

```

Table 1.3: Experiment 3 output

Input	Output
Enter Start Position and End Position: 0 2	30

Experiment 4: Finding max of sliding window from array.

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int a[6] = {2, 6, 7, -12, 76, 9};
    int max = INT_MIN;
    for (int i = 0; i < 4; i++){

        int sum = 0;
        for (int j = 0; j < 3; j++){
            sum = sum + a[i + j];
        }
        if (sum > max)
            max = sum;
    }
    cout << max << endl;
    return 0;
}
```

Table 1.4: Experiment 4 output

73

6. Discussion & Conclusion:

In this lab we have learned the fundamental concepts of arrays and vectors in C++, including basic vector operations, finding the maximum element from a 2D vector, calculating the sum of a subarray, and determining the maximum sum of a fixed-size subarray using the sliding window technique. Some challenges were faced in syntax, maintaining correct indexing and understanding loop boundaries, especially while working with subarrays and sliding windows. Logical errors were harder to detect than syntax errors, which required careful step by step verification. Overall, we have successfully completed all the experiments by doing practical implementation of vector and arrays.

7. Reference:

1. <https://www.geeksforgeeks.org/cpp/cpp-arrays/> accessed at 10.05 PM on 5th February 2026.
2. <https://www.geeksforgeeks.org/cpp/cpp-return-2d-array-from-function/> accessed at 10.35 PM on 5th February 2026.
3. <https://www.geeksforgeeks.org/search/?gq=sliding+window> accessed at 11.15 PM on 5th February 2026.